



Progettazione e Implementazione di un Tool API-Agnostico per la Security Assurance di API REST

Laurea Magistrale in Sicurezza Informatica

Enea Manzi (65935A)

Relatore: Prof. Marco Anisetti

Correlatore: Prof. Claudio Agostino Ardagna

XX Luglio 2026



UNIVERSITÀ
DEGLI STUDI
DI MILANO



Contesto

- Architetture a **microservizi**: la superficie di attacco cresce con le **interazioni tra servizi**, non con il loro numero
- Il **gateway API** come punto di enforcement centralizzato: autenticazione, autorizzazione, rate limiting
- Le vulnerabilità più rilevanti sono **semantiche**: richiedono la conoscenza del **comportamento atteso**
 - 4 delle prime 5 categorie OWASP API Security Top 10 2023



Stato dell'arte e gap

- **G1 Cecità semantica:** scanner e fuzzer operano sulla **sintassi** HTTP, non sulla **semantica** dei contratti
- **G2 Portabilità/profondità:** gli strumenti specializzati richiedono **riscrittura** per ogni target
- **G3 Riproducibilità:** assenza di **determinismo** preclude l'integrazione in pipeline CI/CD
- **G4 Oracle problem:** nessun meccanismo sistematico per costruire **criteri di valutazione** automatici



Obiettivi

Un tool **contract-driven** per la security assurance
di **qualsiasi API REST documentata**: agnostico, sistematico, riproducibile.

- **O1 Agnosticismo applicativo**: OAS e config.yaml come uniche sorgenti a runtime, nessun riferimento hardcoded al target
- **O2 Portabilità con profondità semantica**: superare il compromesso tramite il paradigma contract-driven
- **O3 Riproducibilità deterministica**: output stabile e integrabile in pipeline CI/CD
- **O4 Specified oracles**: criteri di valutazione automatici derivati dalla conoscenza del dominio



Design contract-driven

- **Paradigma contract-driven:** la specifica **OAS** guida la costruzione di probe **semanticamente valide** e delimita la superficie di assessment
- **Agnosticismo applicativo:** le uniche sorgenti di conoscenza del target a runtime sono **openapi.yaml** e **config.yaml**
 - nessun riferimento hardcoded al target nel codice
 - deployabile su **qualsiasi API REST documentata** senza modifiche



Tassonomia: otto domini di security assurance

- **Do API Discovery:** shadow API, inventario endpoint
- **D1 Authentication:** JWT, trasporto credenziali, revoca
- **D2 Authorization:** RBAC, BOLA, segregazione orizzontale e verticale
- **D3 Data Integrity:** coerenza payload, HMAC
- **D4 Availability:** rate limiting, circuit breaker
- **D5 Observability** [0 test attivi]
- **D6 Config & Hardening:** gateway, header HTTP
- **D7 Business Logic:** SSRF, race condition

18 test attivi su **7 domini**; D5 architettralmente preparato, nessun test in questa versione



Box gradient: profondità di assessment

Il livello di ogni test è una **conseguenza delle precondizioni di accesso disponibili**, non una tassonomia imposta dall'esterno.

BLACK_BOX

Nessun prerequisito

Simulazione attaccante esterno
senza credenziali; controlli
perimetrali del gateway

GREY_BOX

Token per ≥ 2 ruoli

Garanzie che presuppongono
stato autenticato; difetti RBAC,
BOLA, segregazione

WHITE_BOX

Admin API del gateway

Ispezione diretta della
configurazione tramite piano di
controllo



Specified oracles: il criterio di valutazione

Come si stabilisce automaticamente se il comportamento osservato è una violazione?

- Il problema dell'oracolo (Barr et al. 2015): generare input è risolto; **valutare il comportamento osservato** è il collo di bottiglia
- APIGuard adotta **specified oracles**: criteri codificati **a priori** nella logica di ogni test, ricavati dalla conoscenza del dominio del controllo
- Sorgenti per dominio:
 - schema OAS → conformità strutturale
 - standard tecnici (NIST, ASVS) → TLS, header
 - configurazione gateway → policy, routing
 - confronto tra ruoli → difetti RBAC, BOLA



Pipeline di esecuzione e connector

- Pipeline a **7 fasi sequenziali**: prime 4 bloccanti (errore interrompe l'avvio), ultime 3 non bloccanti (errori isolati, pipeline prosegue)
- Connector **gerarchici** a 3 livelli: separano la raccolta dei dati grezzi dalla valutazione dell'esito; lo stesso connector funziona in locale, CI/CD e ambienti containerizzati
- Discovery **dinamico**: nuove verifiche vengono registrate automaticamente senza modifiche al core



DAG scheduler e integrazione CI/CD

- Le dipendenze tra test sono dichiarate **esplicitamente** e risolte **topologicamente**; nessun test può essere eseguito prima dei suoi prerequisiti
- Senza DAG, i test RBAC eseguiti prima di quelli di autenticazione producono errori non deterministici invece di fallimenti puliti

Exit codes semantici per CI/CD

- **0 CLEAN** — tutti PASS/SKIP: pipeline prosegue
- **1 FAIL** — almeno un FAIL: blocco merge/deploy
- **2 ERROR** — malfunzionamento interno: revisione manuale
- **10 INFRA** — errore infrastrutturale: diagnosi deployment



Validazione sperimentale

Testbed

- Forgejo 14.0.3 esposto tramite **Kong 3.9 DB-less** in Docker locale
- Scelto per: OAS nativa, autenticazione reale con ruoli multipli, endpoint con gradi diversi di protezione
- Assenza di riferimenti a Forgejo nel codice: **conferma pratica dell'agnosticismo**

Discrepanza rilevata

Forgejo dichiara autenticazione globale; alcuni endpoint genuinamente pubblici rispondono senza

Risultati

Test attivi	18 su 7 domini
PASS	9
FAIL	7
SKIP	2
ERROR	0
Finding totali	98

Idempotenza

- 2 run indipendenti: risultati **byte-identici**



Conclusioni

- **G1** risolto: probe semanticamente valide dalla specifica OAS **raggiungono la logica applicativa**
- **G2** risolto: profondità di assessment **mantenuta su qualsiasi API REST documentata**, senza riscritture
- **G3** risolto: output byte-identico tra run successive, **integrabile via exit code** in CI/CD
- **G4** risolto: specified oracles per ogni verifica, **nessuna interpretazione manuale**

La security assurance diventa un processo **continuo e **misurabile****

- Tassonomia a 8 domini applicabile indipendentemente dall'implementazione
- Approccio sistematico agli specified oracles nel security testing REST
- Assessment engine deterministico basato su DAG, verificato empiricamente



Sviluppi futuri

Estensioni operative

- Adapter per **gateway cloud managed** (AWS API Gateway, Azure APIM): oggi privi di Admin API accessibile, copertura limitata a BB e GB
- Completamento dominio **D5**
Observability
- **Parallelizzazione** intra-batch per ridurre il wall-clock

Direzioni di ricerca

- Piattaforma modulare per **protocolli eterogenei** (gRPC, GraphQL)
- **Formalizzazione** degli oracoli di sicurezza
- **Coverage augmentation** progressiva basata sui risultati precedenti



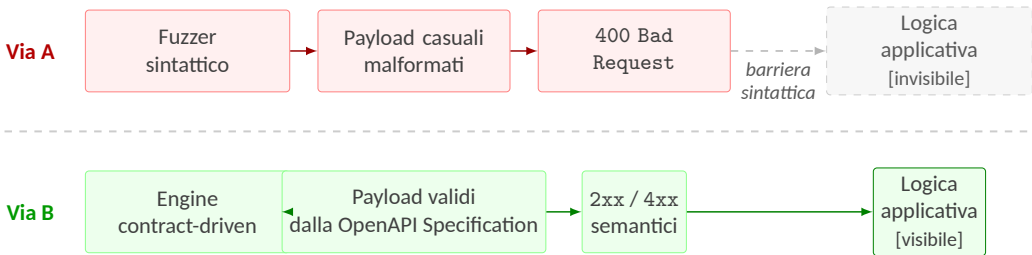
Progettazione e Implementazione di un Tool API-Agnostico per la Security Assurance di API REST

Grazie per l'ascolto! Qualche domanda?

Enea Manzi: enea.manzi@studenti.unimi.it

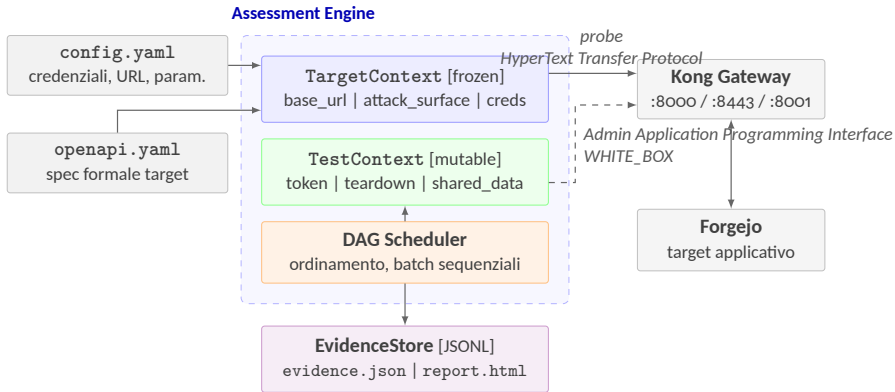


Limite strutturale del fuzzing sintattico



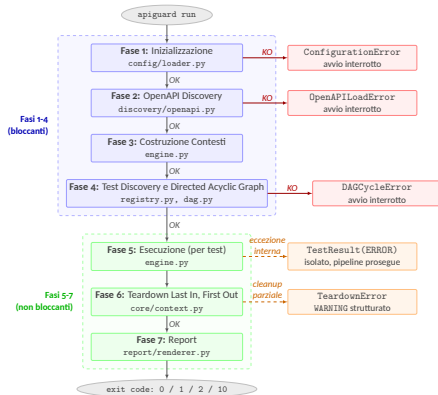


Architettura di APIGuard Assurance



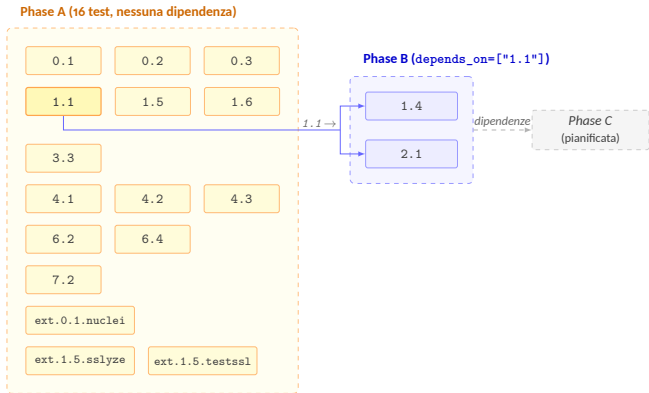


Pipeline a sette fasi





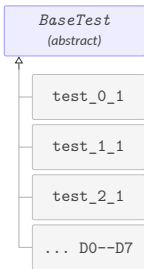
Topologia del Directed Acyclic Graph



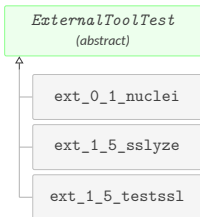


Gerarchia delle classi

Test nativi (DO-D7)

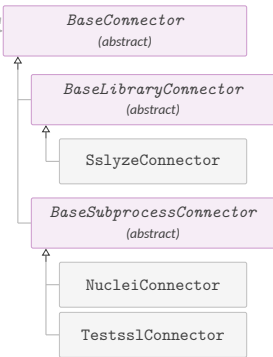


Test esterni



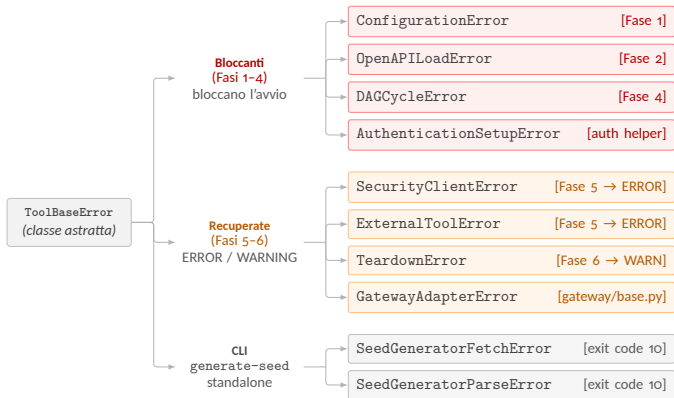
usa

Connettori





Gerarchia delle eccezioni





Struttura interna del TestContext

TestContext [mutable]: tre canali tipizzati con responsabilità distinte

Canale Token	Canale Teardown [LIFO]	Canale Shared Data
ROLE_ADMIN	push(fn_C)	"1.1.token"
ROLE_USER_A	push(fn_B)	"1.1.repos"
ROLE_USER_B	push(fn_A)	"ext.0.1.hits"
<pre>set_token(role, tok) get_token(role) → tok has_token(role) → bool</pre>	<pre>push_teardown(callable) drain order: fn_A → fn_B → fn_C</pre>	<pre>set_shared("id.k", v) get_shared("id.k") → v</pre>
Fase 5: test autenticati scrivono e leggono	Fase 5: push Fase 6: drain inv.	Fase 5: produttore scrive batch successivi leggono

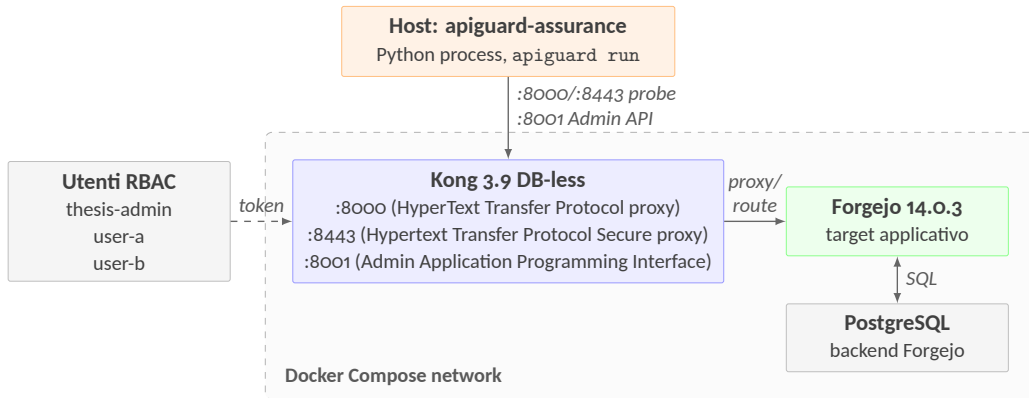


Mappa di copertura



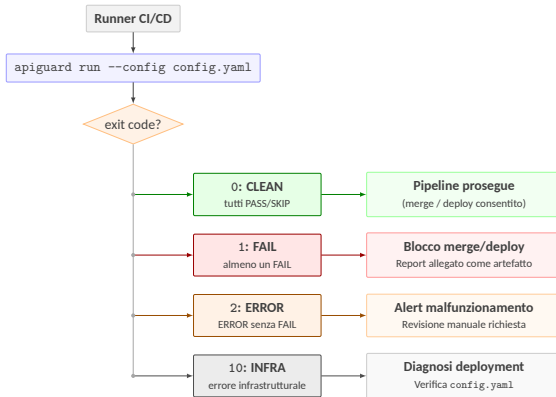


Topologia del testbed



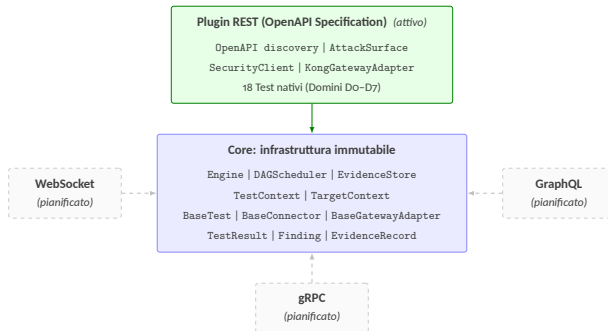


Routing degli exit code





Verso una piattaforma modulare

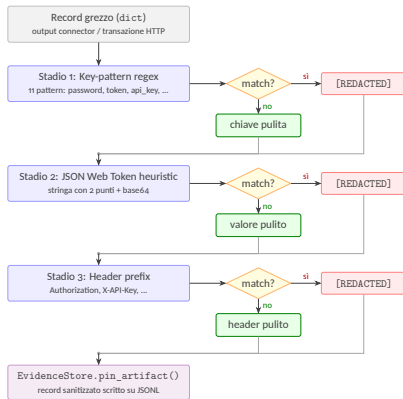


frecce continue: modulo attivo frecce tratteggiate: moduli pianificati

Il discovery dinamico via `pkgutil` permette il caricamento trasparente di moduli di terze parti

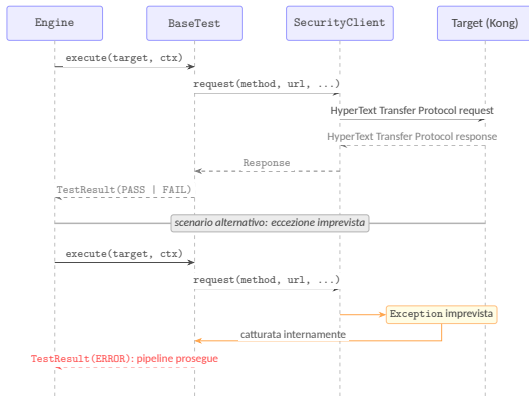


Sanitizzazione a imbuto dell'evidenza





Error isolation in Fase 5





Domini di Assurance

Dominio	Titolo	Ambito di verifica
DO	API Discovery	Inventario degli endpoint esposti; rilevazione di Shadow API non documentate nella specifica.
D1	Identity & Authentication	Meccanismi di riconoscimento del chiamante; robustezza del trasporto delle credenziali; ciclo di vita e revoca dei JSON Web Token.
D2	Authorization	Logiche di controllo degli accessi; difetti di segregazione orizzontale e verticale (Role-Based Access Control, Broken Object Level Authorization).
D3	Data Integrity	Coerenza strutturale dei payload; implementazione delle firme crittografiche (Hash-based Message Authentication Code).
D4	Availability & Resilience	Difese contro l'esaurimento delle risorse; audit di circuit breaker e soglie di rate



Box Gradient

Strategia	Prerequisiti	Razionale
BLACK_BOX	Nessuno (solo accesso di rete)	Controlli perimetrali applicati dal gateway a qualsiasi interlocutore; simulazione di un attaccante esterno senza credenziali.
GREY_BOX	Token per ≥ 2 ruoli distinti	Garanzie che presuppongono stato autenticato con identità di ruolo distinte;



Risultati per test

Test ID	Dominio	Tipo	Status	Finding
0.1	Do: Discovery	Nativo	FAIL	16
0.2	Do: Discovery	Nativo	FAIL	1
0.3	Do: Discovery	Nativo	SKIP	0
1.1	D1: Auth	Nativo	FAIL	74
1.4	D1: Auth	Nativo	PASS	0
1.5	D1: Auth	Nativo	PASS	0
1.6	D1: Auth	Nativo	SKIP	0
2.1	D2: Authorization	Nativo	PASS	0
3.3	D3: Data Integrity	Nativo	PASS	0
4.1	D4: Availability	Nativo	FAIL	2
4.2	D4: Availability	Nativo	PASS	0
4.3	D4: Availability	Nativo	PASS	0



KPI di idempotenza

Key Performance Indicator	Run 1	Run 2	Δ
PASS / FAIL / SKIP / ERROR	9 / 7 / 2 / 0	9 / 7 / 2 / 0	identico
Finding count totale	98	98	identico
pass_rate_pct	56.2%	56.2%	identico
assessment_duration_seconds	290.14 s	290.81 s	+0.23%
Exit code / label	1 / FAIL	1 / FAIL	identico



Performance baseline

Metrica	Run 1	Run 2
Wall-clock elapsed	290.14 s (4:50.14)	290.81 s (4:50.81)
User time	35.09 s	34.67 s
System time	41.51 s	41.62 s
Peak resident set size	287 MB	295 MB



Analisi statica

Tool	Esito	Note
ruff	0 errori	Ruleset E/W/F/I/N/UP/B/S/ANN
mypy	0 errori	Modalità strict: no-implicit-optional, disallow-untyped-defs, warn-return-any
bandit	0 finding severity $\geq$ medium	3 Low (B404, S603x2) soppressi con # noqa documentati: invocazione subprocess controllata per design
vulture	0 finding	Confidence >80%; metodi a



Topologia del DAG (assessment)

Fase DAG	Cardinalità	Test
Phase A (nessuna dipendenza)	16 test	0.1 · 0.2 · 0.3 · 1.1 · 1.5 · 1.6 · 3.3 · 4.1 · 4.2 · 4.3 · 6.2 · 6.4 · 7.2 · ext.0.1.nuclei · ext.1.5.testssl · ext.1.5.sslyze
Phase B (depends_on = ["1.1"])	2 test	1.4 · 2.1



Verdetto di release

Aspetto	Stato	Dettaglio
Analisi statica	✓	0 errori su tutti e cinque gli strumenti
Type safety	✓	0 errori mypy strict; validazione Pydantic attiva su tutti i confini
Documentazione interna	✓	292/292 simboli pubblici con docstring (100%)
Riproducibilità della build	✓	SHA-256 identico su due build consecutive (451 261 byte)
Portabilità	✓	Cold install in ambiente pulito: apiguard version → 0.1.0



Versioni del testbed

Componente	Versione	Ruolo nel testbed
apiguard-assurance	0.1.0	Tool sotto validazione
Python	3.12.3	Interprete del tool
Forgejo	14.0.3 (gitea-1.22.0)	Target applicativo: API REST con specifica OpenAPI nativa
PostgreSQL	15-alpine	Backend di persistenza di Forgejo (dipendenza funzionale, non componente autonomo)
Kong	3.9 DB-less	API Gateway: proxy HyperText Transfer Protocol/Hypertext Transfer Protocol Secure, instradamento verso Forgejo, Admin API su porta 8001
nuclei	3.8.0	Tool esterno per Shadow API discovery (test suite 0-1)



Modelli di esposizione gateway

Modello	Traffico presidiato	Enforcement policy	Accesso al piano di configu- razione	Verifica WHITE_BOX
Gateway Application Programming Interface Dedicato (es. Kong, Apigee)	Nord-sud	Centralizzato (singolo strato)	Admin Application Program- ming Interface diretta	Nativa
Kubernetes Ingress / Gateway Application Programming Interface	Nord-sud	Centralizzato (via K8s API)	kubect1 / Kubernetes Application Program- ming Interface	Controller- dipendente
Service Mesh (es. Istio)	Est-ovest	Distribuito per sidecar proxy	Control plane	N/A (est-ovest)